**Ex No:02** **Date:12-02-26**

**Ex No:1** **Date:05-02-26**

## Introduction to Python Libraries for Data Science

**Aim:**To learn basic Python libraries for Data Science (NumPy, Pandas, Matplotlib, Scikit learn).

**Program:**

11249A248.py

```python
import numpy as np
a=np.array([1,2,3,4,5])
b=np.arange( start_or_stop: 1, stop: 10, step: 2)
c=np.linspace( start: 0, stop: 1, num: 5)
d=np.zeros((2,3))
e=np.ones((2,2))
f=np.eye(3)
g=np.random.randint( low: 1, high: 100,size=(3,3))
print("Array a:",a)
print("arange b:",b)
print("linspace c:",c)
print("zeros d:\n",d)
print("ones e:\n",e)
print("identity f:\n",f)
print("random g:\n",g)
```

OUTPUT:

**Ex No:02** **Date:12-02-26**

```
Run    11249A248    11249A248    1

C:\Users\Admin\PyCharmMiscProject\.ven
Array a: [1 2 3 4 5]
arange b: [1 3 5 7 9]
linspace c: [0.    0.25 0.5  0.75 1.   ]
zeros d:
 [[0. 0. 0.]
 [0. 0. 0.]]
ones e:
 [[1. 1.]
 [1. 1.]]
identity f:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
random g:
 [[95  8 19]
 [52  6 70]
 [80 17 31]]

Process finished with exit code 0
```

11249A248.py

```python
import numpy as np
g=np.random.randint( low: 1, high: 100,size=(3,3))

print("shape:",g.shape)
print("dimensions:",g.ndim)
print("size:",g.size)
print("data type:",g.dtype)

reshaped=g.reshape(9,1)
print("reshaped array:",reshaped)
print("Transpose:\n",g.T)
print("Flattened:",g.flatten())
```

**Ex No:02** **Date:12-02-26**

```
Run    11249A248

C:\Users\Admin\PyCharmMiscProject\.venv\Scripts\python.ex
shape: (3, 3)
dimensions: 2
size: 9
data type: int32
reshaped array: [[81]
 [48]
 [96]
 [78]
 [ 9]
 [92]
 [51]
 [81]
 [ 8]]
Transpose:
 [[81 78 51]
 [48  9 81]
 [96 92  8]]
Flattened: [81 48 96 78  9 92 51 81  8]

Process finished with exit code 0
```

```
11249A248.py
import numpy as np
a=np.array([1,2,3,4,5])

print("Add:",np.add(a, x2: 2))
print("sqrt:",np.sqrt(a))
print("power:",np.power(a, x2: 2))
print("absolute:",np.abs([-1,-2,3]))
```

**Ex No:02** **Date:12-02-26**

11249A248

```
C:\Users\Admin\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\Ad
Add: [3 4 5 6 7]
sqrt: [1.         1.41421356 1.73205081 2.         2.23606798]
power: [ 1  4  9 16 25]
absolute: [1 2 3]

Process finished with exit code 0
```

11249A248.py

```python
import numpy as np
a=np.array([1,2,3,4,5])

print("mean:",np.mean(a))
print("median:",np.median(a))
print("standard deviation:",np.std(a))
print("variance:",np.var(a))
print("minimum:",np.min(a))
print("maximum:",np.max(a))
print("percentile(50):",np.percentile(a, q: 50))
print("sum:",np.sum(a))
```

**Ex No:02** **Date:12-02-26**

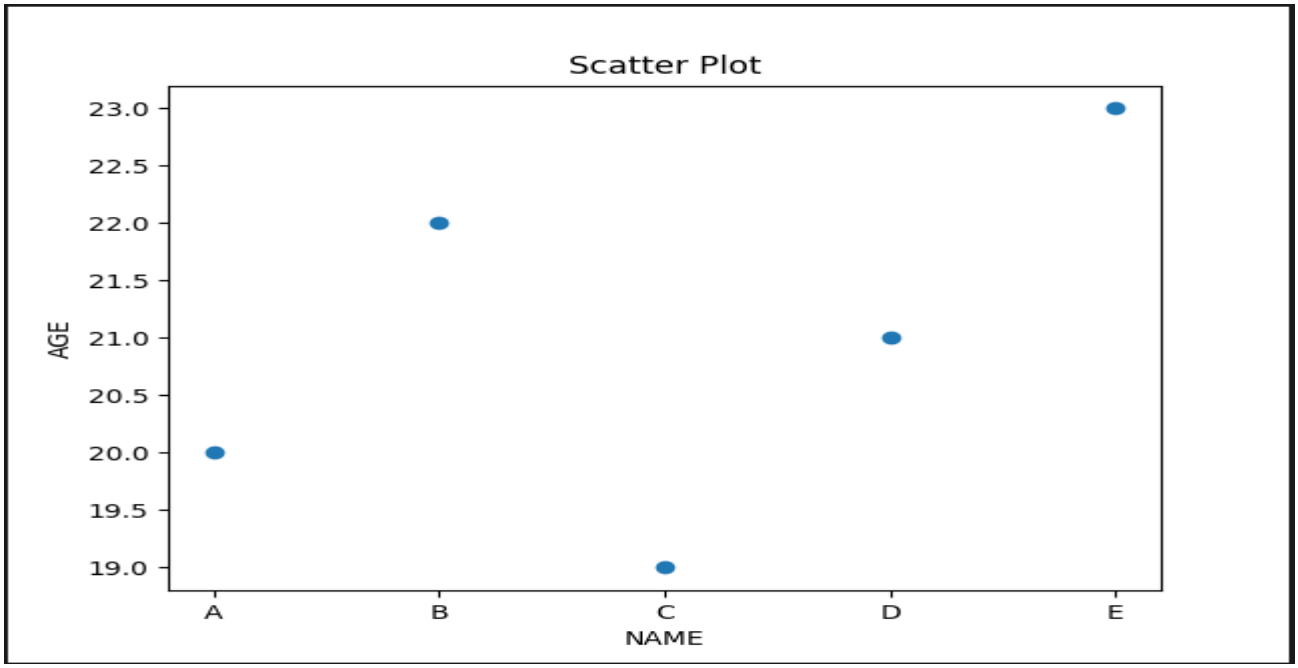
```
11249A248

C:\Users\Admin\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\Ad
Add: [3 4 5 6 7]
sqrt: [1.         1.41421356 1.73205081 2.         2.23606798]
power: [ 1  4  9 16 25]
absolute: [1 2 3]

Process finished with exit code 0
```

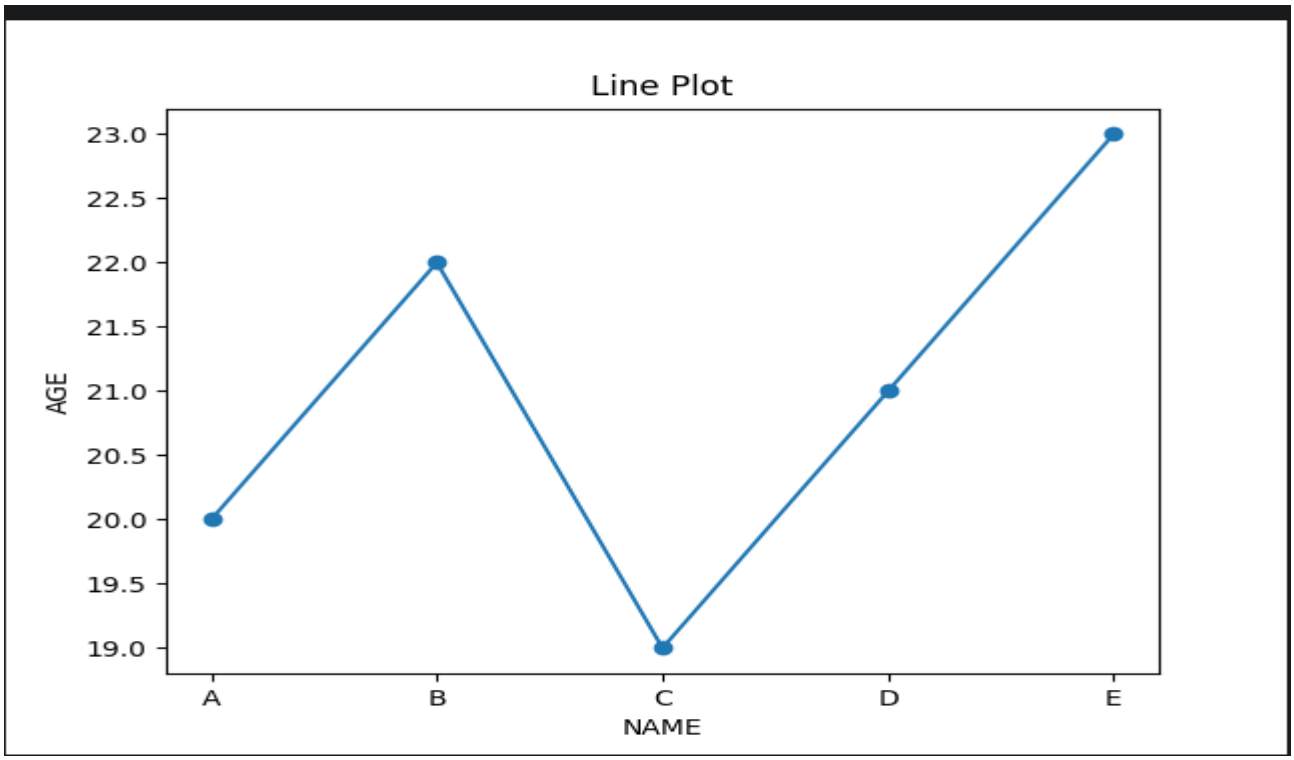
**Ex No:02** **Date:12-02-26**

11249A248.py

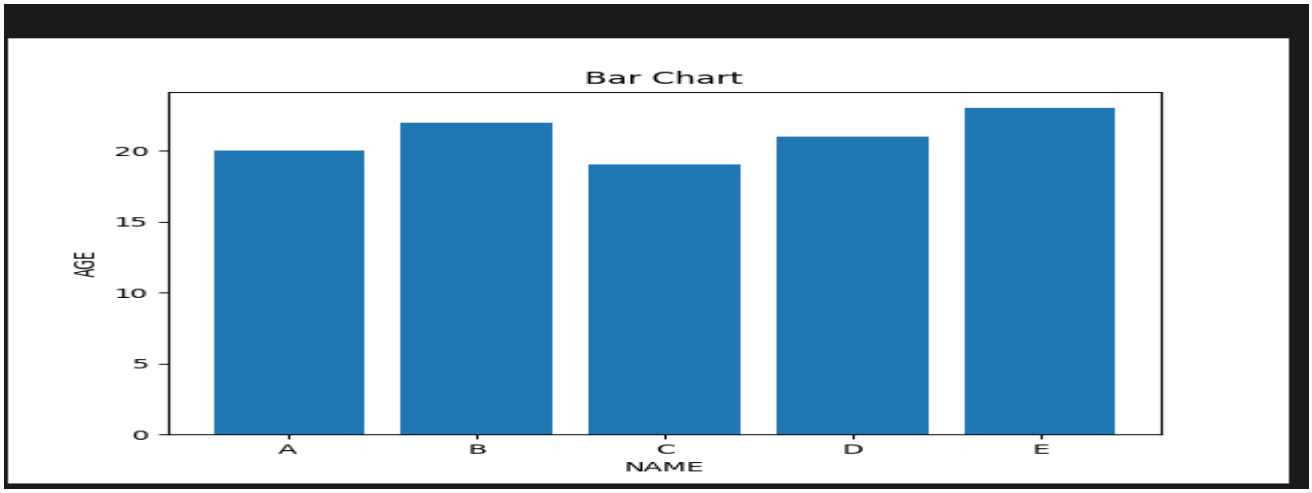
```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data={"NAME":["A","B","C","D","E"],"AGE":[20,22,19,21,23]}
df=pd.DataFrame(data)
plt.scatter(df["NAME"],df["AGE"])
plt.xlabel("NAME")
plt.ylabel("AGE")
plt.title("Scatter Plot")
plt.show()
plt.plot( *args: df["NAME"],df["AGE"],marker="o")
plt.xlabel("NAME")
plt.ylabel("AGE")
plt.title("Line Plot")
plt.show()
plt.hist(df["AGE"],bins=5)
plt.xlabel("Age Range")
plt.ylabel("Count")
plt.title("Histogram")
plt.show()
plt.bar(df["NAME"],df["AGE"])
plt.xlabel("NAME")
plt.ylabel("AGE")
plt.title("Bar Chart")
plt.show()
```

**Ex No:02** **Date:12-02-26**



**Ex No:02** **Date:12-02-26**

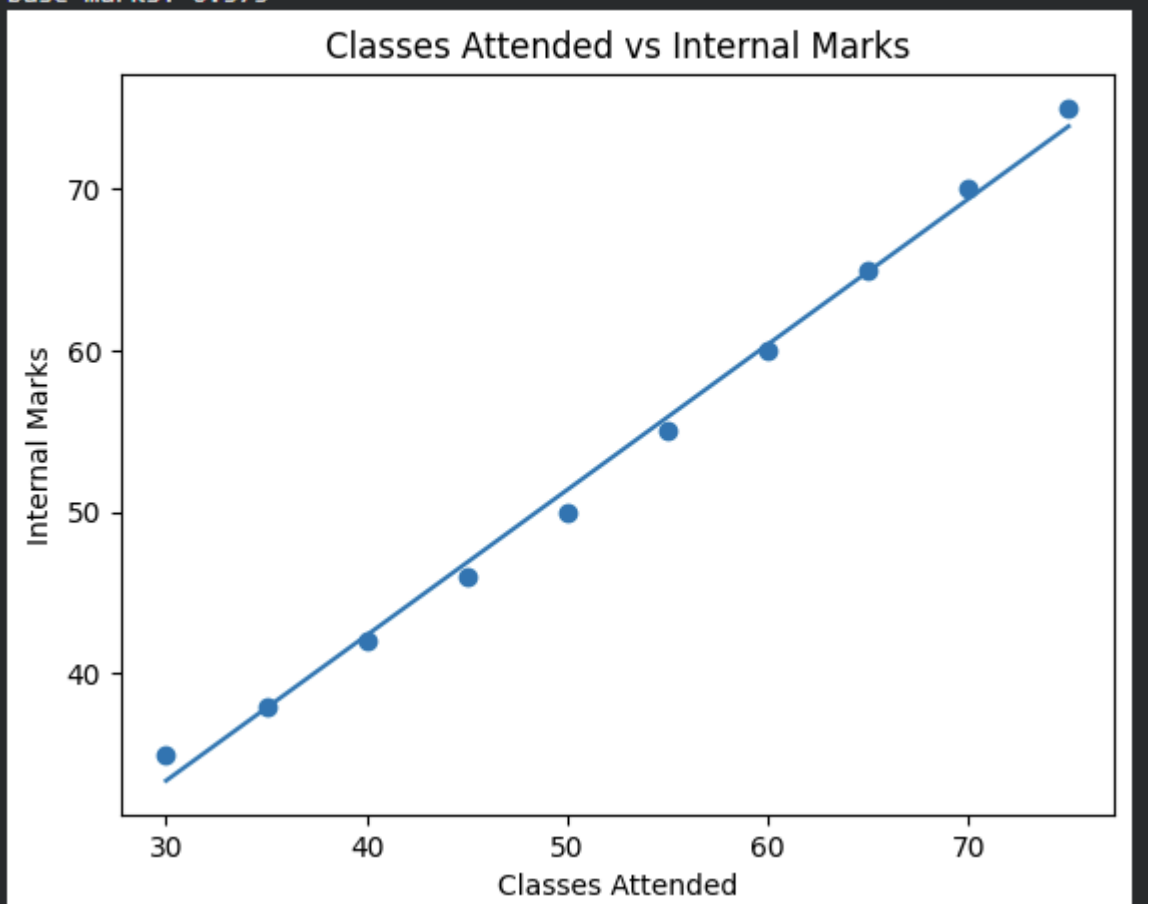



11249A248.py × 11249A248.csv

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
df=pd.read_csv("data_output.csv")
print("Original raw Data")
print(df)
print()
print("missing values count")
print(df.isnull().sum())
print()
df["Age"]=df["Age"].fillna(df["Age"].mode()[0])
df["Salary"]=df["Salary"].fillna(df["Salary"].mode()[0])
print("DATA AFTER CLEANING")
print(df)
print()
df_encoded=pd.get_dummies(df,columns=["Country"])
df_encoded["Purchased"]=df_encoded["Purchased"].map({"Yes":1,"No":0})
print("DATA AFTER ENCODING")
print(df_encoded)
print()
X=df_encoded.drop( labels: "Purchased",axis=1)
y=df_encoded["Purchased"]
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
print("FINAL PROCESSED DATA")
```

**Ex No:02** **Date:12-02-26**

```
Mean Squared Error: 0.578125
marks per class: 0.9
Base marks: 6.375
```




**Result:** Basic operations using these libraries were performed successfully

**Ex No:02** **Date:12-02-26**

**Perform Data exploration and preprocessing in Python**

**Ex No:02** **Date:12-02-26**

**Aim:**To perform data exploration and preprocessing in Python.

**Program:**

11249A248.py × 11249A248.csv

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
df=pd.read_csv("data_output.csv")
print("Original raw Data")
print(df)
print()
print("missing values count")
print(df.isnull().sum())
print()
df["Age"]=df["Age"].fillna(df["Age"].mode()[0])
df["Salary"]=df["Salary"].fillna(df["Salary"].mode()[0])
print("DATA AFTER CLEANING")
print(df)
print()
df_encoded=pd.get_dummies(df,columns=["Country"])
df_encoded["Purchased"]=df_encoded["Purchased"].map({"Yes":1,"No":0})
print("DATA AFTER ENCODING")
print(df_encoded)
print()
X=df_encoded.drop( labels: "Purchased",axis=1)
y=df_encoded["Purchased"]
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
print("FINAL PROCESSED DATA")
```

**Ex No:02** **Date:12-02-26**

OUTPUT:

```
    0   India  22.0  40000.0        No
    1     USA  25.0  60000.0       Yes
... 2   India   NaN  50000.0       Yes
    3     USA  30.0      NaN        No
    4      UK  28.0  72000.0       Yes
    5   India  35.0  58000.0       Yes


    ---missing values count---
    Country      0
    Age          1
    Salary       1
    Purchased    0
    dtype: int64


    ---DATA AFTER CLEANING---
      Country   Age   Salary Purchased
    0   India  22.0  40000.0        No
    1     USA  25.0  60000.0       Yes
    2   India  28.0  50000.0       Yes
    3     USA  30.0  60000.0        No
    4      UK  28.0  72000.0       Yes
    5   India  35.0  58000.0       Yes


    ---DATA AFTER ENCODING---
        Age   Salary  Purchased  Country_India  Country_UK  Country_USA
    0  22.0  40000.0          0           True       False        False
    1  25.0  60000.0          1          False       False         True
    2  28.0  50000.0          1           True       False        False
    3  30.0  60000.0          0          False       False         True
    4  28.0  72000.0          1          False        True        False
    5  35.0  58000.0          1           True       False        False


    ---FINAL PROCESSED DATA(Ready for ai)---
            Age    Salary  Country_India  Country_UK  Country_USA
    0 -1.484615 -1.693214            1.0   -0.447214    -0.707107
    1 -0.742307  0.338643           -1.0   -0.447214     1.414214
    2  0.000000 -0.677285            1.0   -0.447214    -0.707107
    3  0.494872  0.338643           -1.0   -0.447214     1.414214
    4  0.000000  1.557757           -1.0    2.236068    -0.707107
```

**Result:** Basic operations using these libraries were performed successfully

EXPERIMENT 3 DATE:19/3/26

**Implement regularised Linear regression**

**Ex No:02** **Date:12-02-26**

**Aim:**To implement regularized linear regression to improve model performance and control overfitting.

**Program:**

11249A248.py 11249A248.csv

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression,Ridge,Lasso
from sklearn.metrics import mean_squared_error
diabetes=datasets.load_diabetes()
X_train,X_test,y_train,y_test=train_test_split( *arrays: diabetes.data,diabete
lr=LinearRegression().fit(X_train,y_train)
ridge=Ridge(alpha=1.0).fit(X_train,y_train)
lasso=Lasso(alpha=1.0).fit(X_train,y_train)
print("Linear Regression MSE:",round(mean_squared_error(y_test,lr.predict(X
print("Ridge Regression MSE:",round(mean_squared_error(y_test,ridge.predict
print("Lasso Regression MSE:",round(mean_squared_error(y_test,lasso.predict
```

**Result:**The model was successfully implemented, and regularization helped reduce overfitting and improved prediction accuracy.

**Ex No:04** **Date:05-02-26**

**Ex No:02** **Date:12-02-26**

## Implement Naive Bayes classifier for dataset stored as CSV file

### Aim:

To implement the Naive Bayes classifier using a dataset stored in a CSV file for classification.

### Program:

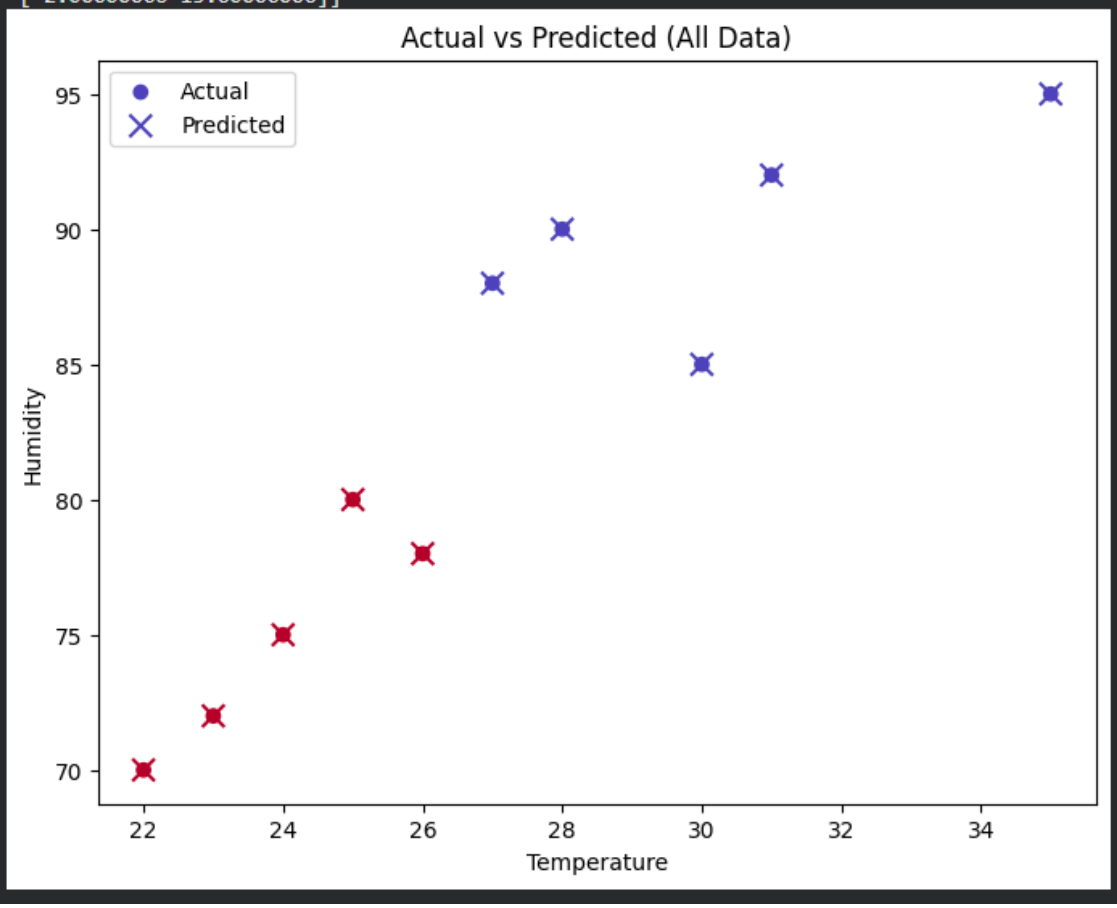
11249A248.py 11249A248.csv

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
data=load_breast_cancer()
X,y=data.data,data.target
X_train,X_test,y_train,y_test=train_test_split( *arrays: X,y,test_size=0.2,rando
features=data.feature_names
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
normal=LogisticRegression(penalty=None,max_iter=5000).fit(X_train,y_train)
normal_acc=accuracy_score(y_test,normal.predict(X_test))
ridge=LogisticRegression(penalty="l2",C=1.0,max_iter=5000).fit(X_train,y_trai
ridge_acc=accuracy_score(y_test,ridge.predict(X_test))
print("Normal Accuracy:",round(normal_acc,3))
print("Ridge(L2) Accuracy:",round(ridge_acc,3))
print("Normal Logistic Coefficients:")
print(pd.Series(np.round(normal.coef_[0], decimals: 3),index=features))
print("Ridge(L2) Coefficients:")
print(pd.Series(np.round(ridge.coef_[0], decimals: 3),index=features))
plt.plot(normal
```

### OUTPUT:

**Ex No:02** **Date:12-02-26**

```
Accuracy: 1.0

class priors:
 [0.375 0.625]

Mean (theta) :
 [[30.66666667 89.33333333]
 [24.         75.        ]]

Variance:
 [[10.88888895 17.55555562]
 [ 2.00000006 13.60000006]]
```




**Result:**

The Naive Bayes model was successfully implemented, and it effectively classified the data with good accuracy.

**Ex No:02** **Date:12-02-26**

**Ex No:02** **Date:12-02-26**

**Ex No:05** **Date:05-03-26**

## Implement regularized logistic regression

### Aim:

To implement regularized logistic regression for classification while reducing overfitting.

### Program:

**Ex No:02** **Date:12-02-26**

11249A248.py 11249A248.csv

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
data=load_breast_cancer()
X,y=data.data,data.target
X_train,X_test,y_train,y_test=train_test_split( *arrays: X,y,test_size=0.2,rando
features=data.feature_names
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
normal=LogisticRegression(penalty=None,max_iter=5000).fit(X_train,y_train)
normal_acc=accuracy_score(y_test,normal.predict(X_test))
ridge=LogisticRegression(penalty="l2",C=1.0,max_iter=5000).fit(X_train,y_trai
ridge_acc=accuracy_score(y_test,ridge.predict(X_test))
print("Normal Accuracy:",round(normal_acc,3))
print("Ridge(L2) Accuracy:",round(ridge_acc,3))
print("Normal Logistic Coefficients:")
print(pd.Series(np.round(normal.coef_[0], decimals: 3),index=features))
print("Ridge(L2) Coefficients:")
print(pd.Series(np.round(ridge.coef_[0], decimals: 3),index=features))
plt.plot(normal
```

OUTPUT:

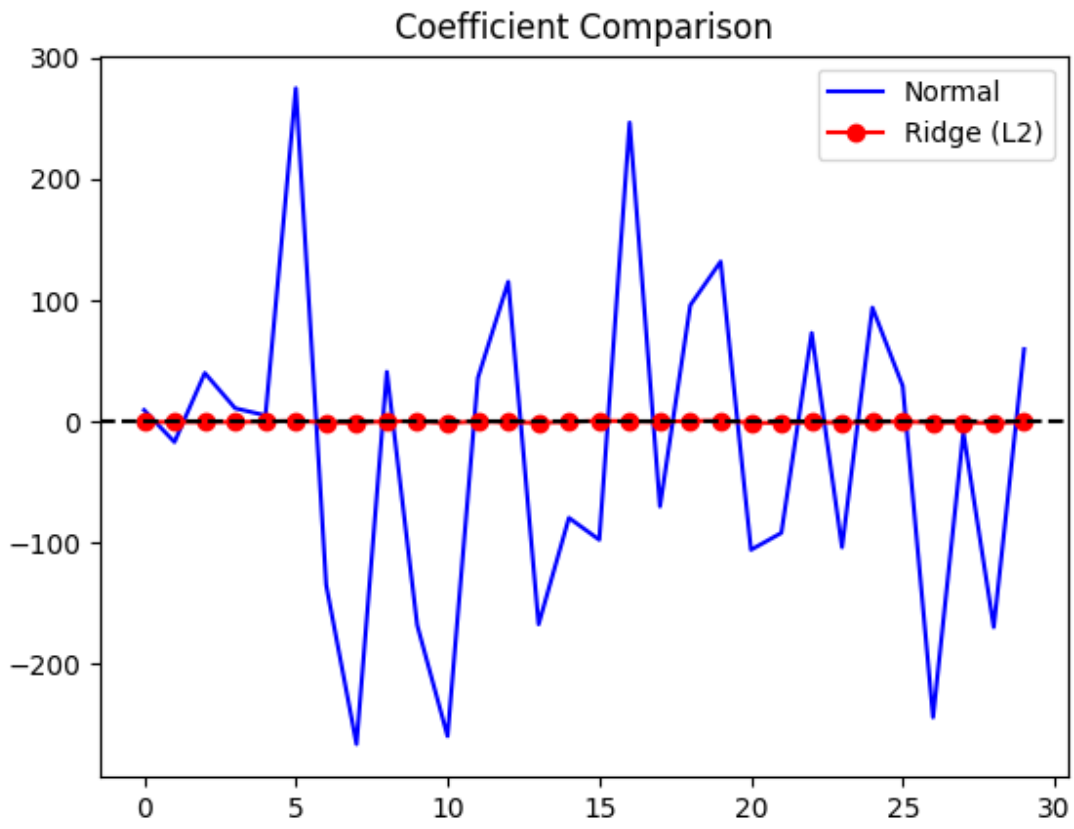
**Ex No:02** **Date:12-02-26**

```
Normal Accuracy: 0.939
Ridge (L2) Accuracy: 0.974

Normal Logistic Coefficients:
mean radius                   9.433
mean texture                 -16.986
mean perimeter                40.021
mean area                     10.890
mean smoothness                5.033
mean compactness             274.940
mean concavity              -134.838
mean concave points         -266.185
mean symmetry                 40.997
mean fractal dimension      -168.018
radius error                -259.639
texture error                 35.918
perimeter error              115.350
area error                  -167.389
smoothness error             -79.540
compactness error            -97.574
concavity error              246.608
concave points error         -70.052
symmetry error                95.954
fractal dimension error      131.939
worst radius                -105.871
worst texture                -91.921
worst perimeter               73.019
worst area                  -103.737
worst smoothness              93.975
worst compactness             29.285
worst concavity             -244.060
worst concave points          -8.908
worst symmetry              -169.655
worst fractal dimension       59.684
```

**Ex No:02** **Date:12-02-26**

```
worst fractal dimension     59.684
dtype: float64

Ridge (L2) Coefficients:
mean radius                -0.432
mean texture               -0.387
mean perimeter             -0.393
mean area                  -0.465
mean smoothness            -0.072
mean compactness            0.540
mean concavity             -0.801
mean concave points        -1.120
mean symmetry               0.236
mean fractal dimension      0.076
radius error               -1.268
texture error               0.189
perimeter error            -0.611
area error                 -0.907
smoothness error           -0.313
compactness error           0.682
concavity error             0.175
concave points error       -0.311
symmetry error              0.500
fractal dimension error     0.616
worst radius               -0.880
worst texture              -1.351
worst perimeter            -0.589
worst area                 -0.842
worst smoothness           -0.544
worst compactness           0.016
worst concavity            -0.943
worst concave points       -0.778
worst symmetry             -1.208
worst fractal dimension    -0.157
dtype: float64
```

**Ex No:02** **Date:12-02-26**




## Result:

The model was successfully implemented, and regularization improved accuracy and prevented overfitting.

**Ex No:02** **Date:12-02-26**

**Ex No:06** **Date:12-03-26**

## Build models using different Ensembling techniques

**Aim:**

To build models using different ensembling techniques to improve prediction performance

PROGRAM

11249A248.py 11249A248.csv

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier,AdaBoostClassifier,StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
iris=datasets.load_iris()
X,y=iris.data,iris.target
X_train,X_test,y_train,y_test=train_test_split( *arrays: X,y,test_size=0.2,random_state=42)
rf=RandomForestClassifier(n_estimators=300).fit(X_train,y_train)
ada=AdaBoostClassifier(n_estimators=100).fit(X_train,y_train)
estimators=[("rf",RandomForestClassifier()),("svc",SVC(probability=True))]
stack=StackingClassifier(estimators=estimators,final_estimator=LogisticRegression()).fit(X_
print("Random Forest Accuracy:",round(accuracy_score(y_test,rf.predict(X_test)),2))
print("AdaBoost Accuracy:",round(accuracy_score(y_test,ada.predict(X_test)),2))
print("Stacking Accuracy:",round(accuracy_score(y_test,stack.predict(X_test)),2))
```

OUTPUT:

**Ex No:02** **Date:12-02-26**

```
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00

Process finished with exit code 0
```

**Result:**

The ensemble models were successfully implemented, and they provided better accuracy compared to individual models.

---

**Ex No:07** **Date:12-03-26**

## Build models using Decision trees

**Aim:**

To build a model using Decision Trees for classification or prediction.

PROGRAM:

**Ex No:02** **Date:12-02-26**

11249A248.py 11249A248.csv

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
iris=datasets.load_iris()
X,y=iris.data,iris.target
X_train,X_test,y_train,y_test=train_test_split( *arrays: X,y,test_size=0.3,random_state=42)
clf=DecisionTreeClassifier(max_depth=3).fit(X_train,y_train)
print("Decision Tree Classifier Accuracy:",round(accuracy_score(y_test,clf.predict(X_test)),
print("Feature Importances:",clf.feature_importances_)
```

OUTPUT:

```
Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0.          0.          0.39426873 0.60573127]

Process finished with exit code 0
```

**Result:**

The Decision Tree model was successfully implemented and provided good prediction performance.

---

**Ex No:08** **Date:02-03-26**

## Build model using SVM with different kernels

**Aim:**

To build an SVM model using different kernel functions for classification.

**Ex No:02** **Date:12-02-26**

**Program:**

11249A248.py 11249A248.csv

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
data=load_iris()
X,y=data.data,data.target
X_train,X_test,y_train,y_test=train_test_split( *arrays: X,y,test_size=0.2,rand
model_linear=SVC(kernel="linear")
model_linear.fit(X_train,y_train)
pred_linear=model_linear.predict(X_test)
acc_linear=accuracy_score(y_test,pred_linear)
print("Linear Kernel Accuracy:",acc_linear)
model_poly=SVC(kernel="poly",degree=3)
model_poly.fit(X_train,y_train)
pred_poly=model_poly.predict(X_test)
acc_poly=accuracy_score(y_test,pred_poly)
print("Polynomial Kernel Accuracy:",acc_poly)
model_rbf=SVC(kernel="rbf")
model_rbf.fit(X_train,y_train)
pred_rbf=model_rbf.predict(X_test)
acc_rbf=accuracy_score(y_test,pred_rbf)
print("RBF Kernel Accuracy:",acc_rbf)
```

OUTPUT

```
Linear Kernel Accuracy: 1.0
Polynomial Kernel Accuracy: 0.9666666666666667
RBF Kernel Accuracy: 1.0

Process finished with exit code 0
```

**Ex No:02** **Date:12-02-26**

**Result:**

The SVM models were successfully implemented with various kernels, and they achieved good classification performance

**Ex No:09** **Date:02-03-26**

**Implement K-NN algorithm to classify a dataset**

**Aim:**To implement the K-Nearest Neighbors (K-NN) algorithm to classify a dataset

**Program:**

**Ex No:02** **Date:12-02-26**

11249A248.py 11249A248.csv

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score,confusion_matrix,classification_r
data=pd.read_csv("mobile_price.csv")
X=data.drop( labels: "price_range",axis=1)
y=data["price_range"]
X_train,X_test,y_train,y_test=train_test_split( *arrays: X,y,test_size=0.2,rand
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train,y_train)
y_pred=knn.predict(X_test)
print("Accuracy:",accuracy_score(y_test,y_pred))
print("Confusion Matrix:\n",confusion_matrix(y_test,y_pred))
print("Classification Report:\n",classification_report(y_test,y_pred))
```

OUTPUT:

```
Accuracy: 1.0
Confusion Matrix:
 [[1]]
Classification Report:
              precision    recall  f1-score   support

           2       1.00      1.00      1.00         1

    accuracy                           1.00         1
   macro avg       1.00      1.00      1.00         1
weighted avg       1.00      1.00      1.00         1
```

**Result:**

**Ex No:02** **Date:12-02-26**

The K-NN model was successfully implemented and it classified the dataset with good accuracy

**Ex No:10** **Date:02-03-26**

## Build model to perform Clustering using K-means after applying PCA and determining

### Aim:

To build a clustering model using K-Means after applying PCA for dimensionality reduction and determining optimal clusters.

PROGRAM:

**Ex No:02** **Date:12-02-26**

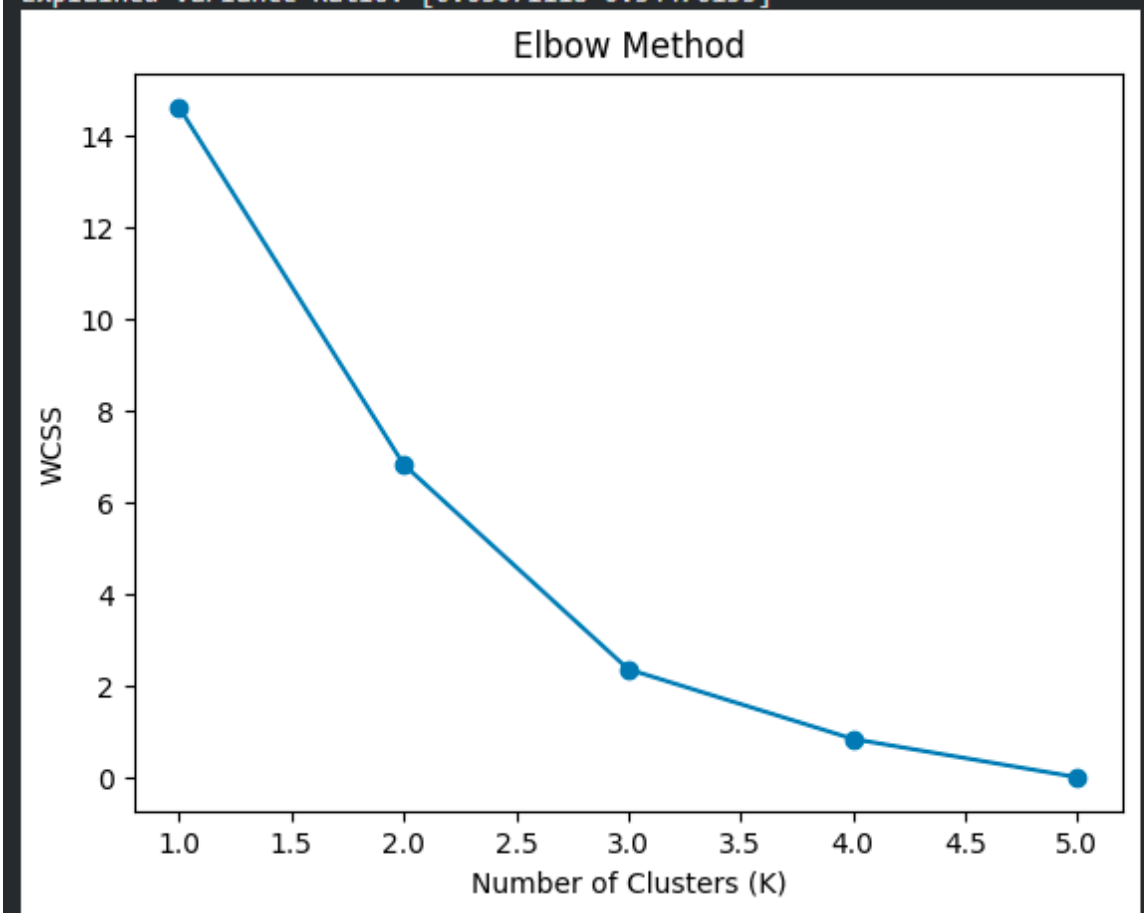
11249A248.py 11249A248.csv

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
data=pd.read_csv("Mall_Customers.csv")
X=data[["Age","Annual Income (k$)","Spending Score (1-100)"]]
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
pca=PCA(n_components=2)
X_pca=pca.fit_transform(X_scaled)
print("Explained Variance Ratio:",pca.explained_variance_ratio_)
wcss=[]
max_k=min(10,len(X_pca))
for i in range(1,max_k+1):
    kmeans=KMeans(n_clusters=i,random_state=0,n_init=10)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
plt.plot( *args: range(1,max_k+1),wcss,marker="o")
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()
optimal_k=min(3,len(X_pca))
kmeans=KMeans(n_clusters=optimal_k,random_state=0,n_init=10)
clusters=kmeans.fit_predict(X_pca)
```

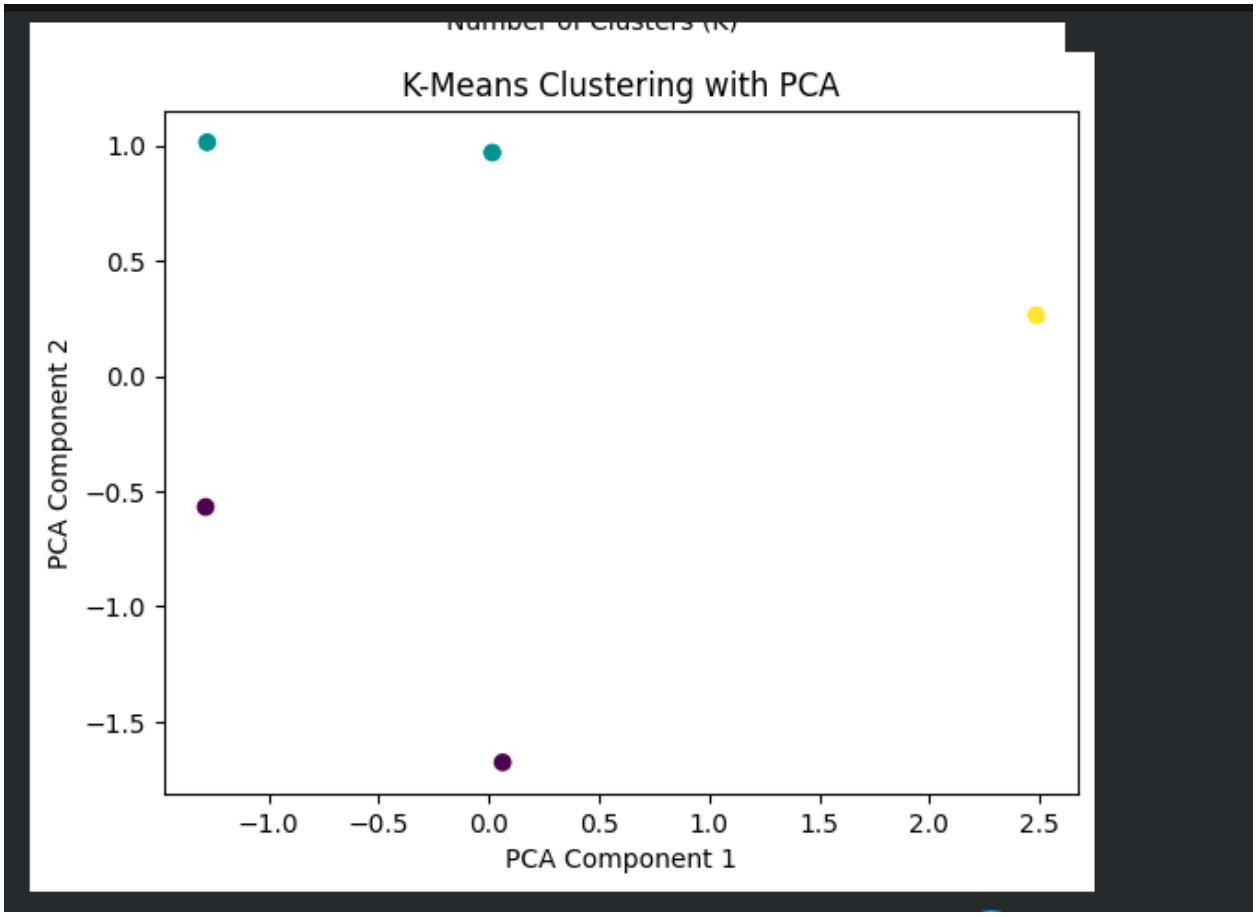
OUTPUT:

**Ex No:02** **Date:12-02-26**

```
Explained Variance Ratio: [0.63071118 0.34476153]
```



**Ex No:02** **Date:12-02-26**




**Result:**

The K-Means clustering model was successfully implemented after PCA, and the data was grouped effectively into clusters